

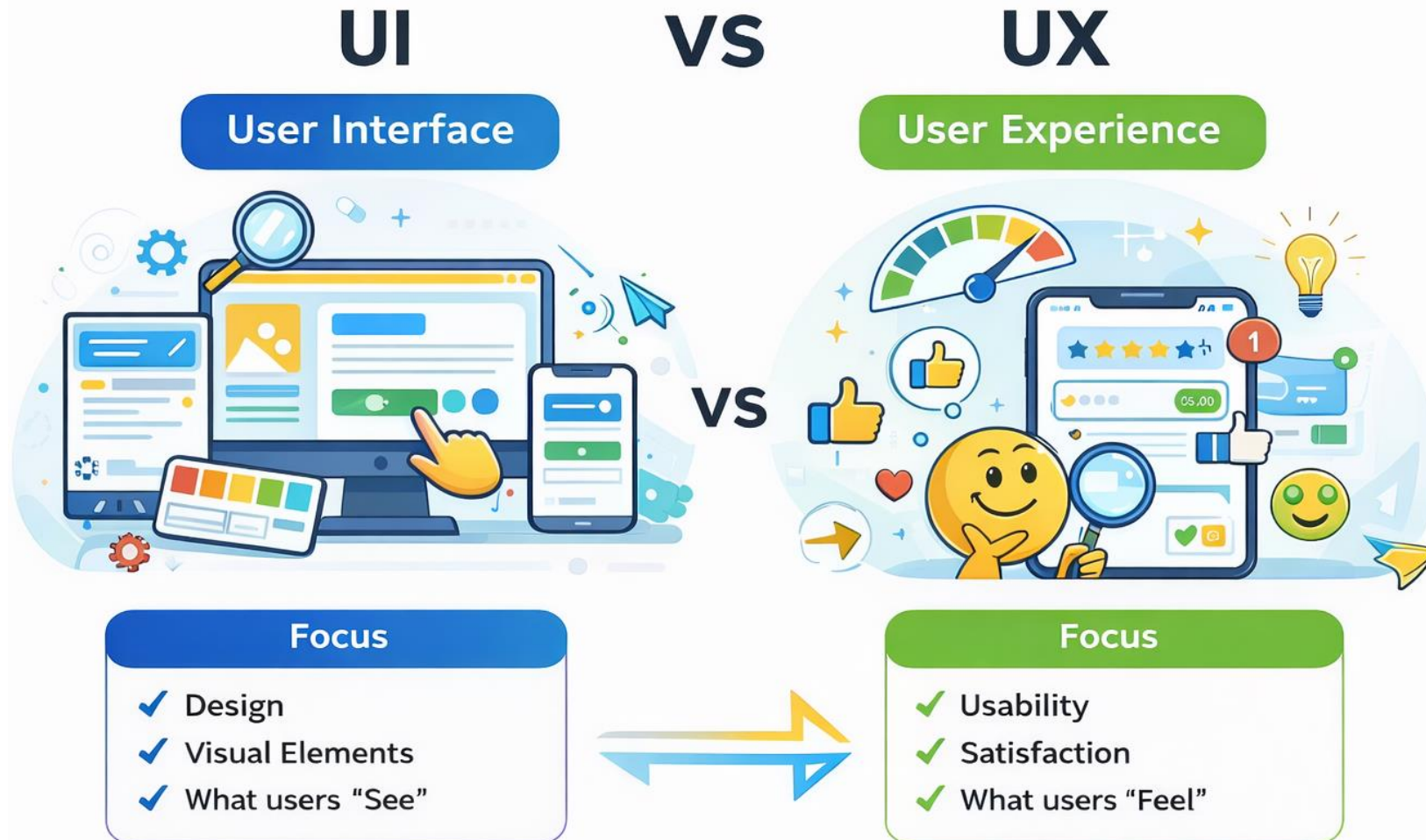
IT 관련 용어

IT 관련 용어



UI vs UX

UI와 UX에 대하여 살펴 보겠습니다.



≡ UI(User Interface)

사용자가 직접 보고 조작하는 **화면 요소**를 의미합니다.

사용 예시

버튼(Button)
메뉴(Menu)
입력창(Input Box) 디자인
색상, 폰트, 레이아웃

핵심 포인트

디자인 중심
시각적 요소
사용자가 "보는 것"

≡ UX(User Experience)

사용자가 서비스를 이용하면서 느끼는 전체 경험을 의미합니다.

사용 예시

버튼이 잘 눌리는가?
페이지가 빠르게 로딩되는가?
사용 흐름이 자연스러운가?
사용하기 쉬운가?
에러 메시지가 이해하기 쉬운가?

핵심 포인트

사용성 중심
만족도, 편의성
사용자가 "느끼는 것"

Client

자원 요청을 하는 주체를 말하는 데, 일반적으로 웹 브라우저입니다.



Server

서버는 특수성과 전문성을 위하여 일반 컴퓨터보다 좀더 높은 사양이 요구되는 컴퓨터입니다.

서버의 종류

서버(Server)란 이름 그대로 무엇인가를 서비스해주는 것을 말합니다.

예를 들어서, 메일의 송신/수신을 담당해 주는 메일 서버, 웹 사이트의 기능을 제공해 주는 웹 서버, 데이터를 저장하고 있는 데이터 베이스 서버 등입니다.



메일 서버



웹 서버



데이터 베이스 서버

서버의 종류

항목	설명	예시
웹 서버	HTML/이미지/동영상 등의 자원에 대한 웹 사이트의 기능을 제공	Apache, Nginx 등등
메일 서버	메일의 송수신과 메일 수신을 위한 서버	Sendmail, Postfix 등등
데이터 베이스 서버	데이터를 검색해주는 서버	Oracle, MySql, Sqlite 등등
파일 서버	파일을 저장하여 공유하고자 할 때 사용하는 서버	Samba 등등
FTP 서버	파일을 업로드/다운로드 해주는 서버	

대표적인 웹 Server

Apache와 Nginx는 웹 요청을 처리해 HTML, 이미지, API 응답 등을 사용자에게 전달하는 소프트웨어입니다.

Apache HTTP Server

개요

가장 오래되고 널리 사용된 웹 서버
Apache Software Foundation에서 개발
안정성과 확장성이 뛰어나 기업에서 많이 사용

장점

설정이 직관적이고 기능이 풍부
다양한 모듈 지원 (PHP, Python 등과 연동 쉬움)
전통적인 웹 환경에 강함

단점

동시 접속자가 많아지면 메모리 사용량 증가
대규모 트래픽 처리에서는 상대적으로 비효율



Nginx

개요

가볍고 빠른 웹 서버 (최근 매우 인기)
러시아 개발자 Igor Sysoev가 개발

장점

매우 높은 성능 (특히 정적 파일 처리)
동시 접속 처리에 강함 (대규모 서비스에 적합)
메모리 사용량이 적음

단점

Apache보다 설정이 다소 어려움
.htaccess 미지원 (중앙 설정만 가능)



≡ 데이터 표현법

≡ 데이터 표현법

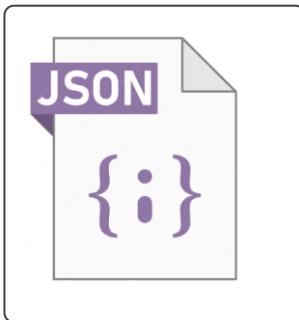
웹에서 표현할 수 있는 데이터 표현법에는 다음과 같은 항목들이 있습니다.

JSON

가장 많이 사용

가볍고, JavaScript/React에서 사용 편리

```
{  
  "id": 1,  
  "name": "노트북",  
  "price": 1000000  
}
```



XML

태그 기반

과거 시스템에서 많이 사용

```
<product>  
  <id>1</id>  
  <name>노트북</name>  
  <price>1000000</price>  
</product>
```



HTML

화면 표시용

브라우저 렌더링

```
<h1>노트북</h1>  
<p>가격: 1000000원</p>
```

TEXT

가장 단순한 문자열

상품명: 노트북, 가격: 1000000원

☰ 관련 용어

☰ 관련 용어

다음은 REST API에서 많이 언급이 되는 용어입니다.

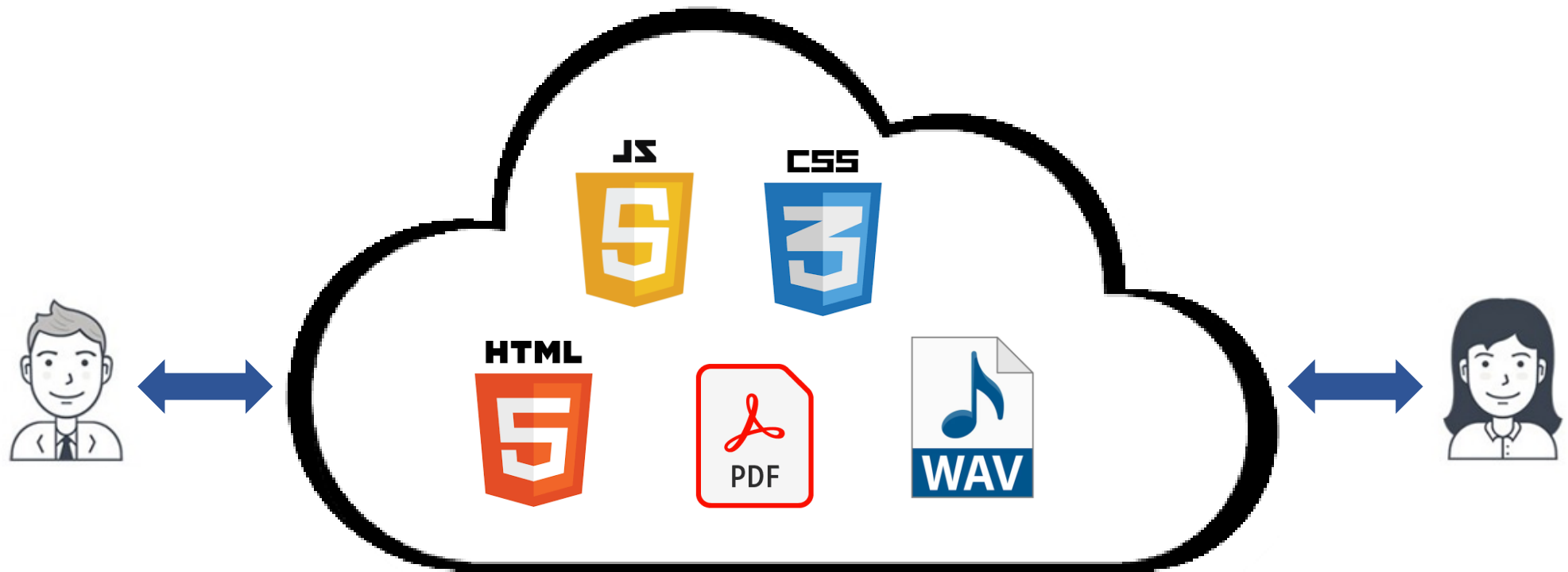
관련 용어

항목	설명
API(Application Programming Interface)	소프트웨어 시스템과 통신하기 위해 따라야 하는 규칙을 정의해 놓은 명세를 의미합니다. 예시) 자바 API, 파이썬 API, 판다스 API
State	자원의 실제 데이터 상태 정보(JSON, XML, HTML, CSS, TEXT 등)를 의미합니다.
Resource	웹 서버에 존재하는 HTML 페이지, 소리 파일, 이미지 파일, pdf 파일 등등을 의미합니다.
REpresentation	서버가 클라이언트에게 응답해주는 Resource의 State 표현 방식입니다.(JSON, XML 등)
REST(Representational State Transfer)	자원에 대한 접근을 시도할 때 이름을 구분자로 구분 지어서 자원의 정보를 주고 받는 동작을 의미합니다. API 작동 방식에 대한 조건을 부과하는 소프트웨어 아키텍처 자체를 의미합니다. 2000년 로이 필딩의 박사 학위 논문에서 처음 소개되었습니다.
REST API	REST 기반으로 서비스 API를 구현한 것을 의미합니다. 공공 기관에서 제공하는 OpenAPI는 누구나 접근이 가능하며, 거의 대부분 REST API를 제공해 줍니다.
RESTful Service	REST API를 제공하는 웹 서비스를 의미하는 용어입니다.

≡ 자원(Resource)

≡ 자원(Resource)

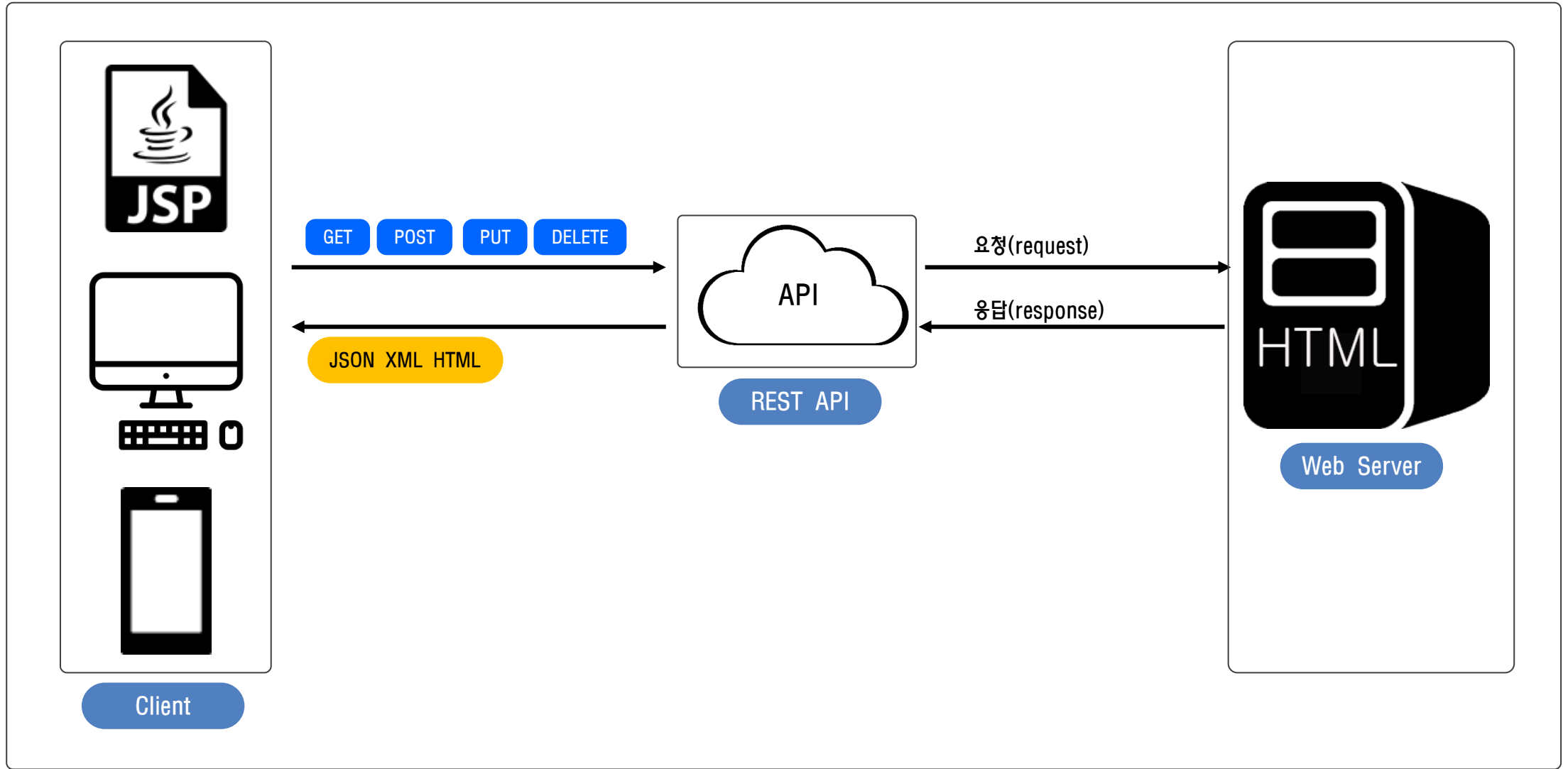
Client와 Server가 서로 주고 받을 수 있는 데이터를 의미합니다.



☰ 웹 프로그래밍

☰ 웹 프로그래밍 구성 요소

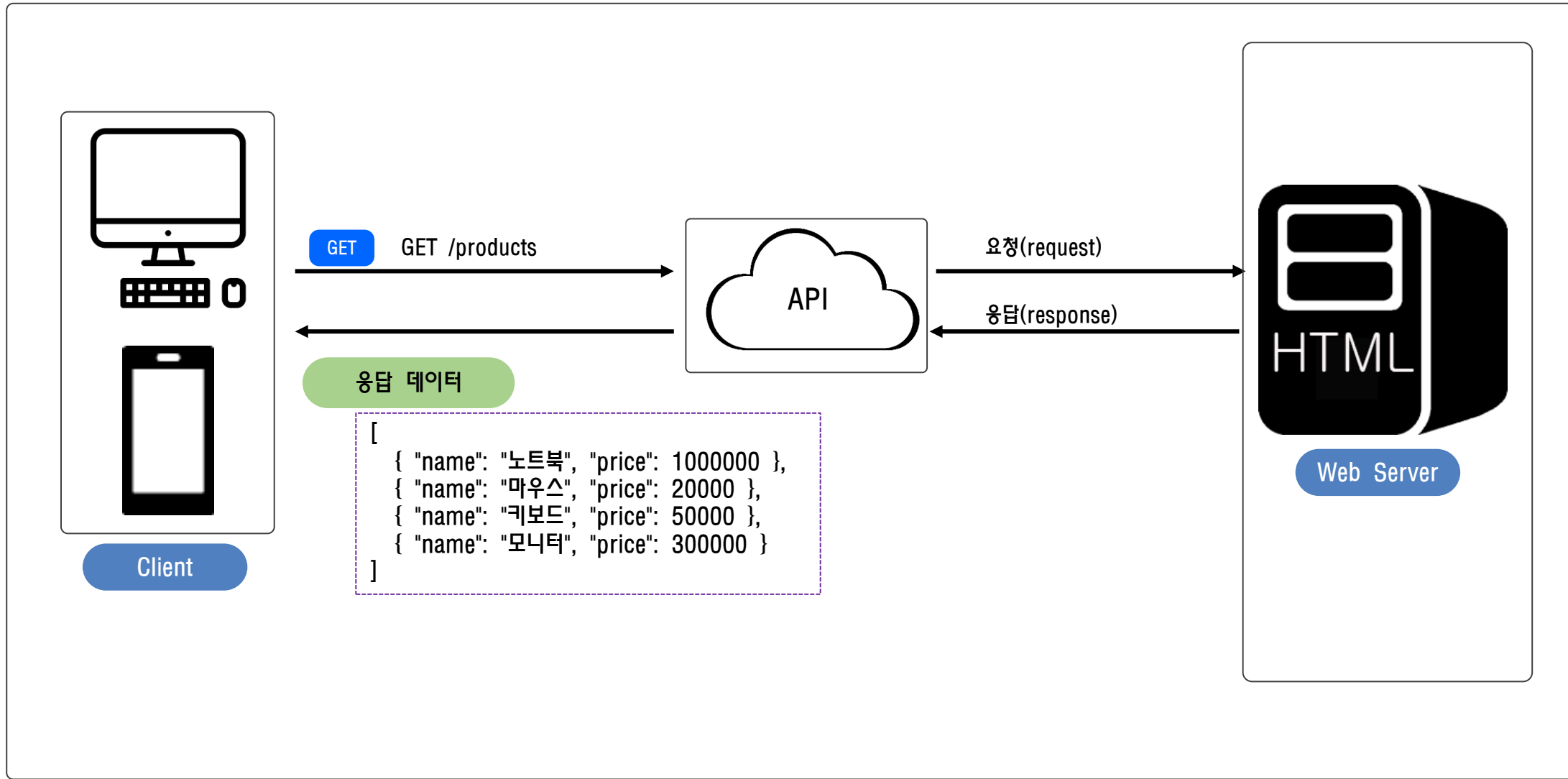
웹 프로그램은 클라이언트가 자원(Resource) 요청을 하고 웹서버가 그에 대한 응답을 해주는 구조로 되어 있습니다.



API

Application Programming Interface

어떤 프로그램이 다른 프로그램의 기능을 사용할 수 있도록 제공해주는 인터페이스(접점)입니다.



library&framework

library&framework

라이브러리와 프레임워크에 대하여 살펴 봅니다.

library 특징

필요할 때 내가 가져다 쓰는 도구 모음

개발자가 직접 코드의 흐름을 제어
필요한 기능만 골라서 호출(call)해서 사용
독립적으로 사용 가능

사용 예시

NumPy → 수치 계산
Pandas → 데이터 분석
jQuery → DOM 조작

framework 특징

정해진 틀 안에서 내가 코드를 넣어 사용하는 구조

프레임워크가 전체 흐름을 제어
개발자는 정해진 규칙에 따라 코드를 끼워 넣음
구조가 이미 만들어져 있음

사용 예시

Spring Boot → 백엔드 서버 개발
Django → 웹 서비스 개발
Angular → 프론트엔드

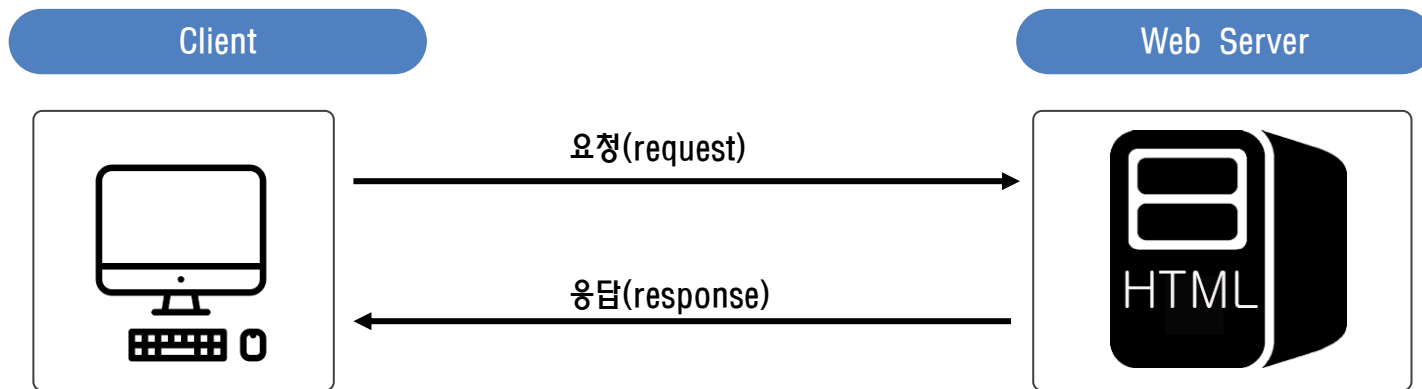
항목	Library	Framework
제어 흐름	개발자	프레임워크
사용 방식	필요할 때 호출	구조 안에 맞춰 개발
자유도	높음	낮음
구조	없음 (자유)	있음 (강제됨)
예	NumPy, jQuery	Spring Boot, Django

≡ REST

≡ REST

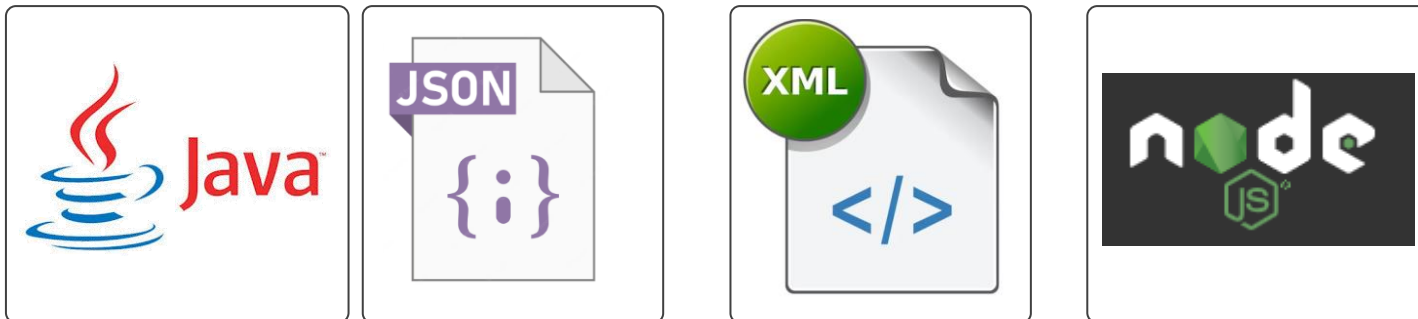
서버가 클라이언트로 전송하는 Resource를 여러 가지 형태로 전달 할 수 있다는 개념으로 사용됩니다.

CS 환경의 요청/응답



Resource의 다양한 형태

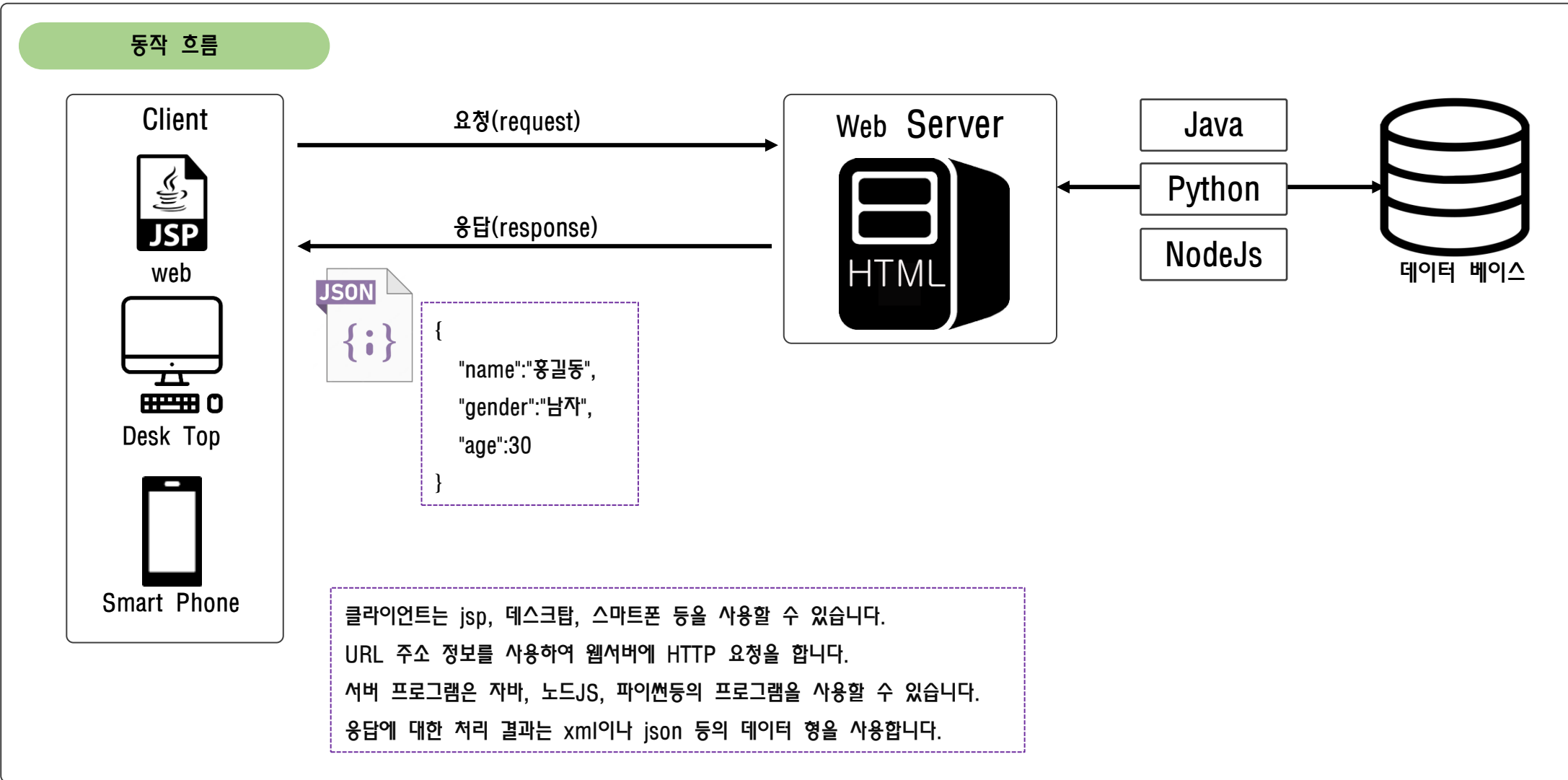
Resource는 다음과 같이 다양한 형태로 클라이언트에게 전송될 수 있습니다.



REST

REST

다음은 REST API의 사용 예시입니다



Rest의 장단점

Rest의 장점과 단점은 다음과 같습니다.

장점

이미 검증된 기술(서블릿, jsp, 스프링 등)에서 사용 가능

HTTP 프로토콜을 사용하는 플랫폼에서 사용 가능

REST API 메시지의 명확성(가독성 높음)

인증/보안/성능/백업/분산 처리 등의 이전 기술 재활용 가능

단점

표준이 존재하지 않음

활용 가능한 메소드가 4가지

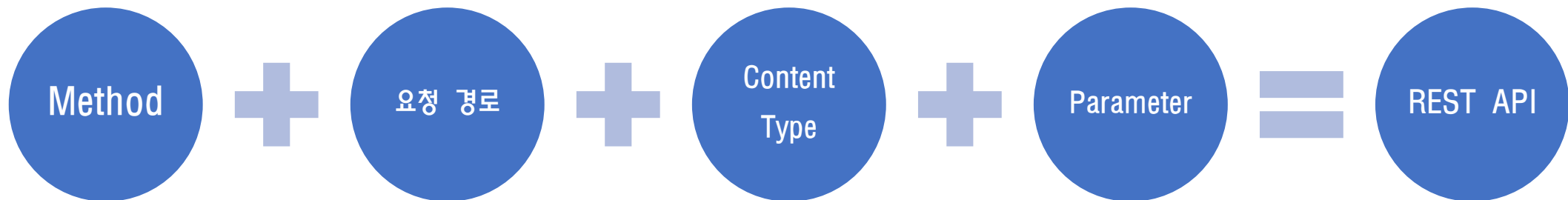
구형 브라우저에서 사용 불가능

REST

구성 요소

다음은 Rest에서 사용되는 여러 가지 요소에 대한 설명입니다.

URL를 이용하여 자원(Resource)을 명시하고, HTTP Method를 사용하여 자원에 대한 CRUD 동작을 적용시키는 기술을 의미합니다.



구성 요소

항목	설명
Method	어떤 유형의 메소드 요청(get, post, put, delete 등등)을 처리할 것인가를 지정합니다.
요청 경로	어떠한 요청에 대하여 동작을 할 것인가를 지정합니다.
응답 콘텐츠	클라이언트로 응답해 줄 콘텐츠의 유형(text, html, json, xml 등등)을 지정합니다.
파라미터	클라이언트가 요청한 데이터 파라미터와 경로를 위한 파라미터를 의미합니다.

CRUD

CRUD가 무엇인지 살펴 보도록 합니다.

CRUD(출처 위키 백과)

CRUD는 컴퓨터 소프트웨어가 가지는 기본적인 데이터 처리 기능인 Create(생성), Read(읽기), Update(갱신), Delete(삭제)를 묶어서 일컫는 말입니다. 사용자 인터페이스가 갖추어야 할 기능(정보의 참조/검색/갱신)을 가리키는 용어로서도 사용이 됩니다.



이름	조작	SQL
Create	생성	INSERT
Read(또는 Retrieve)	읽기(또는 인출)	SELECT
Update	갱신	UPDATE
Delete(또는 Destroy)	삭제(또는 파괴)	DELETE

Method & CRUD

Method는 '동작(행위)'을 의미하고, CRUD는 '데이터 처리 방식'을 의미합니다.

HTTP Method를 통해 Resource를 처리하도록 설계된 Architecture를 의미합니다.

웹 사이트의 이미지, 텍스트, DB 내용 등의 모든 자원에 고유한 ID인 HTTP URI를 부여합니다.

대표적인 메소드는 POST, GET, PUT DELETE 등이 있습니다.

Method와 CRUD 매핑

HTTP Method	CRUD	설명
GET	Read	데이터 조회
POST	Create	데이터 생성
PUT	Update	전체 수정
PATCH	Update	일부 수정
DELETE	Delete	데이터 삭제

예시 (상품 API)

GET /products	→ 상품 목록 조회
GET /products/1	→ 상품 1개 조회
POST /products	→ 상품 생성
PUT /products/1	→ 상품 전체 수정
PATCH /products/1	→ 상품 일부 수정
DELETE /products/1	→ 상품 삭제

Protocol

컴퓨터나 네트워크 장치끼리 통신하기 위한 약속 또는 규칙을 의미합니다.

Protocol



사람이 대화할 때 문법과 순서가 있듯이, 컴퓨터도 데이터를 주고 받을 때 정해진 형식과 절차가 있습니다.

이를 프로토콜(Protocol)이라고 하며 컴퓨터나 네트워크 장치끼리 통신하기 위한 약속 또는 규칙을 의미합니다.

이러한 약속은 반드시 지켜야만 서로 다른 장치나 프로그램이 문제 없이 통신할 수 있습니다.

Protocol

컴퓨터나 네트워크 장치끼리 통신하기 위한 약속 또는 규칙을 의미합니다.

프로토콜 예시

항목	설명
HTTP	웹 브라우저와 웹 서버가 페이지를 주고 받는 규칙. 주소창에 http:// 로 시작하는 프로토콜입니다.
HTTPS	HTTP에 보안(암호화) 기능이 추가된 프로토콜입니다. 요즘 대부분의 웹사이트가 https:// 프로토콜을 사용합니다.
FTP	파일을 전송할 때 사용하는 프로토콜입니다.
SMTP / IMAP / POP3	이메일을 주고 받는 프로토콜입니다.
TCP / UDP	데이터 전송 방식을 정의한 프로토콜입니다. TCP는 안정적(순서 보장), UDP는 빠름(실시간 스트리밍에 사용).

IP Address

네트워크에 연결된 장치(컴퓨터, 스마트폰, 서버 등)를 구분하기 위한 고유한 주소입니다.



192.168.0.1

IP Address

A unique address to identify devices connected to a network (computer, smartphone, server, etc.), using numbers separated by dots (e.g., **192.168.0.1**).

IP Address

A unique address to identify devices connected to a network (computer, smartphone, server, etc.), using numbers separated by dots (e.g., **192.168.0.1**).

IP Versions

● IPv4 (Internet Protocol version 4)

- 32-bit address with approx. **4.3 billion** unique addresses
- Format: **192.168.0.1**

● IPv6

- Introduced due to shortage of IPv4 addresses

192.168.0.1

- Format: **192.168.0.1**

≡ REST

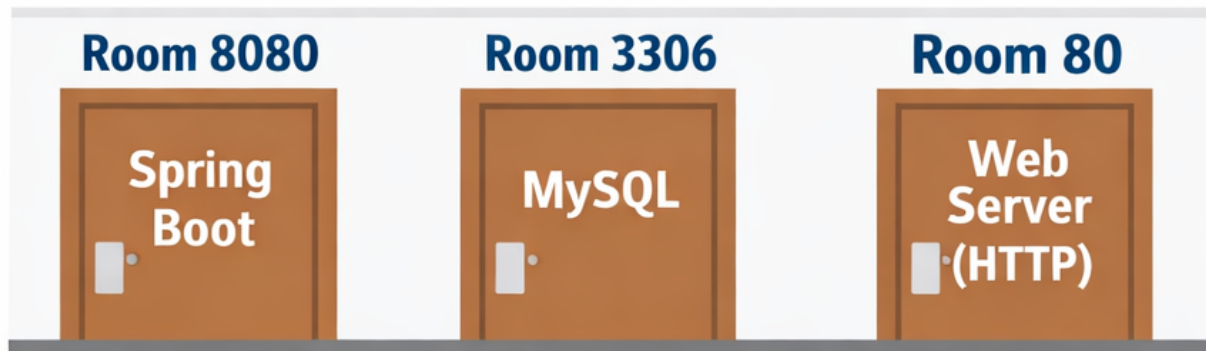
≡ Port

하나의 서버에서 여러 서비스를 동시에 실행하기 위해 구분하는 정수형 숫자입니다.

Port Number: *A number to distinguish programs on a server*



IP Address = Building



- 8080 → Spring Boot
- 3306 → MySQL
- 80 → Web Server (HTTP)

≡ well-known ports

다음은 우리가 많이 사용하는 잘 알려진 포트(well-known ports)라고 합니다.

잘 알려진 포트

포트 번호	서비스	내용
80	HTTP	웹 송수신
443	HTTPS	웹 송수신
22	SSH	SSH 통신
1521	Oracle Database	오라클 데이터 베이스 통신
3306	MySql	MySql 데이터 베이스 통신

≡ REST

≡ localhost

localhost는 내 컴퓨터 자신을 가리키는 주소 (127.0.0.1)를 의미합니다.

그림 설명

내 컴퓨터(localhost)에서 9000번 포트에서 동작 중인 Spring Boot와 3000번에서 동작 중인 React에 대한 이미지 입니다.

On my localhost (my computer)



<http://localhost:9000>

Running on **Port 9000**



<http://localhost:3000>

Running on **Port 3000**

웹 페이지 주소 형식

웹 페이지의 주소 형식은 다음과 같이 구성되어 있습니다.

get 전송 방식에 대하여 웹 페이지의 주소 형식을 세부적으로 살펴 보겠습니다.

get 방식의 주소 형식

http://localhost:9000/MyWeb/02.parameter/to.jsp?id=hong&password=abc1234&mpoint=100

http://	localhost:	9000	/MyWeb	/02.parameter/to.jsp	?	id=hong&password=abc1234&mpoint=100
프로토콜	호스트이름	웹서버포트번호	컨텍스트	요청커맨드	?	파라미터리스트
			URI			
URL					?	쿼리 문자열

웹 페이지 주소 형식

클라이언트에서 서버로 넘겨 주는 파라미터들의 모음을 Query 문자열(String)이라고 부릅니다.

웹 프로그램을 만들다 보면 페이지 간에 정보의 교환이 필요한 경우가 있는데 이를 위해서 웹프로그래밍이 제공하는 기술이 Query 문자열입니다.

get 방식의 주소 형식

id	=	hong	&	password	=	abc1234	&	mpoint	=	100
이름		값		이름		값		이름		값

항목	설명
정의	클라이언트에서 서버로 넘겨 주는 일정 형식의 데이터(? 이후의 정보들)
? 기호	url과 쿼리 문자열(파라미터 리스트)의 구분자
& 기호	파라미터 목록들의 구분자
= 기호	파라미터의 이름과 값을 구분하기 위한 구분자입니다.
사용 예시	네이버 접속시 만약 아이디가 abcd, 비밀번호가 1234이라고 한다면 http://www.naver.com?id=abcd&password=1234 인데, 여기서 id=abcd&password=1234를 Query 문자열이라고 합니다.
특징	공백은 +으로 변환한다 한글은 % 기호와 함께 16진수로 변환이 되어 전달됩니다. 참고 사항 : RFC2396 규약
선수 지식	<form> 태그의 이해 http의 get / post 방식의 이해

≡ HTTP 프로토콜

HTTP 프로토콜은 인터넷 통신 프로토콜(protocol) 중 실제 사용자와 연결된 최상위 프로토콜(Protocol)입니다.

Protocol이란 네트워크 통신을 수행하기 위한 전송 규약을 의미합니다.

예를 들어서 파일을 업로드 및 다운로드 하고자 하는 경우에는 FTP(File Transfer Protocol)라는 기술을 사용합니다. 이 또한 Protocol의 한 종류입니다.

항목	설명
HTTP	www 서비스를 제공하기 위한 통신 규약 입니다. 웹 서비스를 제공하는 Hyper Text Transfer Protocol입니다. 웹 서버와 클라이언트 상호간에 HTTP를 통하여 통신하기 위한 규약입니다.
Stateless (상태 보존 無)	웹 서버는 클라이언트의 상태 값을 기억하지 못합니다. 상태 값이란 이전의 어떠한 설정 정보를 말합니다. 예를 들어서 이전 로그인시 사용했던 id, 비밀번호 등을 기억하지 못합니다.
특징	지속적인 서비스를 원활히 하지 못합니다.(단점) 하지만 오버헤드는 줄일 수 있습니다.(장점)
도입된 개념	이러한 단점을 보완하기 위한 개념이 세션과 쿠키입니다. 쿠키(cookie)는 ' 클라이언트 ' 영역에 정보를 저장하는 개념입니다. 세션(session)는 ' 서버 ' 영역에 정보를 저장하는 개념입니다.

HTTP 프로토콜

요청과 응답

HTTP 프로토콜은 크게 요청(Request) 영역과 응답(Response) 영역으로 구분되어 있습니다.



항목	설명	요청(Request)	응답(Response)
시작 라인	요청인가요? 응답인가요?	시작(요청) 라인	시작(응답) 라인
헤더	수행 날짜, 서버의 정보, 브라우저 버전 등등	헤더(header) 일반 헤더 / 응답 헤더	헤더(header) 일반 헤더 / 요청 헤더
본문	요청이나 응답에 필요한 내용	본문(body) 요청 (메시지)	본문(body) 요청 (메시지)

항목	설명
요청 메시지 (Request)	http method : get / post 중 택일(기본 값 : get 방식) 접근하고자 하는 URL 파라미터 : form태그 내의 넘겨 줘야 할 항목들
응답 메시지 (Response)	상태 코드(200, 404, 500 등등) 컨텐츠의 타입 정보(텍스트, 그림, html 등등) 컨텐츠의 실제 내용(html 문서, 이미지 등등)

스타일 시트와 자바 스크립트 코딩시
그림과 같은 코드를 본적이 있습니다.
이 또한 역시 Content Type라고 봐도 무방합니다.

```
<style type="text/css">  
</style>  
<script type="text/javascript">  
</script>
```

HTTP 프로토콜

요청과 응답

요청(Request)을 하기 위한 전송 방식은 크게 Get 방식과 Post 방식이 있습니다.

항목	Get 방식	Post 방식
시작 라인	Get abc.jsp?id=hong&password=1234	Post abc.jsp
요청 헤더	Host : www.naver.com User-Agent : Mozilla/5.0	Host : www.naver.com User-Agent : Mozilla/5.0
본문	-	id=hong&password=1234
특징	파라미터 정보가 url 뒤에 붙어서 전송이 됩니다. Post 방식에 비하여 빠릅니다.	길이에 제약이 없습니다. 보안에 유리합니다.

응답(Response)

항목	설명
응답 헤더 (header)	HTTP/1.1 200 OK ... Content-Type : text/html ...
몸체 (body)	<html> ... </html>

HTTP Status 404 - /JqueryProject/A01.concept/abc.jsp

type Status report

message /JqueryProject/A01.concept/abc.jsp

description The requested resource is not available.

실제로 존재하지 않은 파일에 대하여
호출을 하게 되면 그림이 같이
404 응답 값을 리턴해줍니다.

Content-Type은 Mime Type이라고
부르기도 하며 이 문서는 메모장으로
볼 수 있는 html 문서 형식입니다.
라는 정도의 의미로 해석할 수 있습니다.

0.42

≡ 서버의 응답 코드(Status Code)

HTTP 서버 응답의 첫 번째 행은 클라이언트 요청의 성공 여부와 이유를 표시합니다.

응답 코드

상태는 숫자 3개로 된 서버 응답 코드와 설명 메시지로 주어집니다.

항목	설명
100대 : 정보	요청을 받아 처리 중
200대 : 성공	클라이언트의 요청이 성공적으로 처리되었음을 알려 줍니다.
300대 : 경로 변경	요청을 완전히 처리하기 위해 추가적인 액션 수행 301 : sendRedirect를 사용하게 되면 header 정보에 들어가는 상수입니다. 302 : 강제 페이지 이동할 때 많이 사용하는 코드입니다.
400대 : 클라이언트 에러	클라이언트의 요청에 오류가 있는 경우에 발생합니다. 파일 발견이 되지 않는 경우에 발생합니다. 사용자의 오타로 인하여 페이지를 찾을 수 없습니다.
500대 : 서버 에러	클라이언트의 요청에 오류가 있는 경우에 발생합니다. 문법적인 오류로 인하여 발생합니다. 사용자의 오타로 인한 경우에 발생합니다.

이벤트(Event)

이벤트 개요

HTTP 서버 응답의 첫 번째 행은 클라이언트 요청의 성공 여부와 이유를 표시합니다.

응답 코드

항목	설명
Event Driven Programming	특정 이벤트에 따라서 실행 코드를 작성하는 기술을 의미합니다.
Event 이름	click, load, unload 등등입니다. Event Type라고 불리기도 합니다.
Event 속성	on + load (on + 이벤트 이름)
이벤트 핸들러 (Event Handler)	이벤트 속성에 할당이 되어 있는 함수 [어떤 어떠한 이벤트가 발생하면... ~해라.] 라는 식으로 해석합니다.
Event의 종류	키보드, HTML 프레임, 유저 인터페이스, 구조 변화, 터치 이벤트

예를 들어서, 마우스 버튼을 누르면 **mousedown** 이벤트가 발생하고,
마우스 버튼을 떼면 **mouseup** 이벤트가 발생한 다음 바로
click 이벤트가 발생합니다.

CORS(교차 출처 리소스 공유)

웹 보안을 유지하면서 다른 출처의 리소스에 접근할 수 있도록 해주는 메커니즘입니다.

Origin 구성 요소

Origin은 다음과 같은 요소로 구성이 되어 있습니다.

항목	설명
URI Schema	http, https
Hostname	localhost(127.0.0.0), naver.com, daum.net 등등
포트 번호	80, 9000, 3000 등등

REST

CORS

CORS의 기본 정책은 사이트 A에서 사이트 B의 자원을 읽어올 수 없습니다.

SOP(Same-origin policy)

SOP은 동일한 Origin(포트 번호)에만 요청을 보낼 수 있도록 하는 정책을 의미합니다.

다음 구성 요소 중에서 하나라도 동일하지 않으면 SOP 정책에 위반이 되므로, 자원(resource)을 공유할 수 없습니다.

왜냐하면, Origin이 상이(相異)하므로 CORS 정책(policy)에 의하여 블록킹(blocked by) 되기 때문입니다.

다음 예시에서 포트 번호 3000인 React에서 포트 번호 9000인 Spring Boot에는 접근이 불가능합니다.



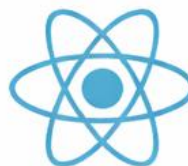
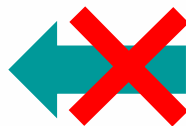
http://localhost:9000

Running on Port 9000

자원 A1

자원 A2

자원 A3



React

http://localhost:3000

Running on Port 3000

자원 B1

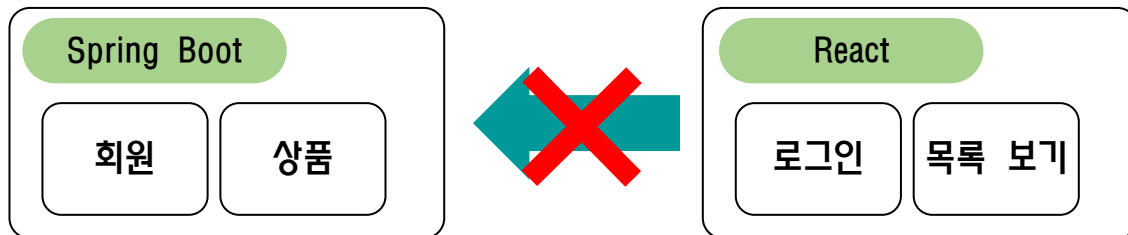
자원 B2

자원 B3

CORS policy 오류 문제

CORS의 기본 정책은 사이트 A에서 사이트 B의 자원을 읽어올 수 없습니다.

CORS 오류 예시



다음은 React(포트 번호 : 3000)에서 Spring Boot(포트 번호 : 9000)로 Json 데이터를 요청했을 때 반환되는 오류 메시지입니다.
이 문제를 해결하려면 우선 CORS 정책에 대한 이해가 필요합니다.

```
localhost/:1 Access to XMLHttpRequest at 'http://localhost:9000/react/data/exam01'
from origin 'http://localhost:3000' has been blocked by CORS policy:
No 'Access-Control-Allow-Origin' header is present on the requested resource.
xhr.js:244
GET http://localhost:9000/thymeleaf/rest/ex01 net::ERR_FAILED 401
```

CORS의 필요성

Origin이 달라도, 상호 요청을 주고 받을 수 있게 정해 놓은 표준 스펙입니다.

CORS가 필요한 이유

CORS 설정은 서버 측에서 이루어지며, 서버는 응답 헤더인 "Access-Control-Allow-Origin"을 설정하여 특정 도메인이나 모든 도메인으로부터의 요청을 허용할 수 있습니다.

내용	설명
보안 강화	자신이 서비스하는 동일한 출처의 리소스에만 접근할 수 있도록 제한을 시키고, 다른 외부 출처의 악성 스크립트로부터 보호됩니다.
자원 공유	다른 도메인이나 포트에서 호스팅되는 리소스에 접근하는 경우 CORS를 통해 서로 다른 출처 간에 리소스를 공유할 수 있습니다. 예를 들어, React에서 스프링 애플리케이션의 데이터를 사용하고자 하는 경우, 스프링은 CORS를 허용하여 요청이 제대로 이루어질 수 있도록 해주어야 합니다.
확장성	CORS를 통해 다른 출처의 리소스에 접근하고 상호 작용할 수 있으므로, 웹 애플리케이션의 기능과 서비스의 다양성과 확장성을 향상시킬 수 있습니다.

기본 방식과 Rest 방식

REST API를 본격적으로 설명하기 전에 이전 방식과의 차이점을 간략히 살펴 봅니다.

HTTP을 이용하여 데이터를 전송하는 방식에는 Get 방식과 Post 방식이 존재합니다

그리고, 웹 페이지 하나에 대한 표현 방식은 크게 URL(기능) 영역과 파라미터(데이터)영역으로 구분이 됩니다.

다음 예시는 쇼핑을 의미하는 `/shopping`와 관련 파라미터들을 의미하는 `command=boList&bo=1`으로 구분이 되고 있습니다.

웹 페이지 표현

`/shopping?command=boList&bo=1`
URI(기능) 영역 + 파라미터(데이터) 영역

Rest 방식은 URI(기능) 영역과 파라미터(데이터)영역을 합쳐서 1개의 URI 형태로 표현하는 기법입니다.

다음 예시는 쇼핑물의 1번 게시물을 표현하고 있습니다.

Rest 표현 방식

`/shopping/boList/1`
URI(기능) 영역

REST API 설계 원칙

REST API의 설계 원칙에 대하여 살펴 보도록 하겠습니다.

설계 원칙

규칙	좋은 예시(good)	나쁜 예시(bad)
URI는 '/'로 끝나지 않아야 합니다.	http://localhost:9000/members	http://localhost:9000/members/
동사 형식보다는 명사 형식을 사용합니다.	http://localhost:9000/members http://localhost:9000/products	http://localhost:9000/getMemberList http://localhost:9000/getAllProduct
단수형 보다는 복수형 명사를 사용합니다.	http://localhost:9000/members	http://localhost:9000/member
URI는 기본 소문자로 사용합니다.	http://localhost:9000/members	http://localhost:9000/MEMBERS
언더스코어(_) 대신에 하이픈(-)을 사용합니다.	http://localhost:9000/member-view	http://localhost:9000/member_view
집합형 자료는 복수 명사를 사용합니다.	http://localhost:9000/boards http://localhost:9000/products	http://localhost:9000/board http://localhost:9000/allProduct
관계 표현 형태에 의한 하위 리소스 사용하기	http://localhost:9000/students/it/김철수	-
쿼리 파라미터 사용 금지	http://localhost:9000/products/delete/100	http://localhost:9000/products?action=delete&no=100
파일 확장자는 URI에 포함하지 않습니다.	-	-

사용 예시

다음은 각각의 요청에 대한 사용 예시입니다.

사용 예시

전송 방식	url 패턴	설명
Get	/	메인 페이지로 이동합니다.
Get	/members/new	회원 가입 페이지로 이동합니다.
Post	/members/new	유효성 검사를 수행 후 회원 가입을 진행합니다.
Get	/members/login	로그인 페이지로 이동합니다.
Get	/members/login/error	로그인 실패시에 대한 처리 후 로그인 페이지로 이동합니다.
Get	/members/logout	로그 아웃을 수행합니다.(시큐리티에서 정의됩니다.)
Get	/admin/products/new	상품 등록 페이지로 이동합니다.
Post	/admin/products/new	상품 등록 버튼을 클릭하여 상품 등록을 합니다.
Get	/admin/product/{productId}	상품 수정 페이지로 이동합니다.
Post	/admin/product/{productId}	상품 수정 버튼을 클릭하여 상품 수정을 합니다.
Get	/admin/products 또는 /admin/products/{page}	페이징 처리를 이용하여 관리자용 상품 목록 보기 화면을 구현합니다. page 파라미터는 옵션 사항입니다.
Get	/product/{productId}	메인 화면에서 상품 클릭시 상세 페이지로 이동합니다.

사용 예시

다음은 각각의 요청에 대한 사용 예시입니다.

사용 예시

전송 방식	url 패턴	설명
Post	/order	상품 상세 페이지에서 [주문하기] 버튼을 클릭합니다.
Get	/orders 또는 /orders/{page}	구매 이력을 조회합니다.
Post	/order/{orderId}/cancel	해당 주문 번호에 대하여 주문을 취소합니다.
Post	/cart	해당 상품을 장바구니에 저장합니다.
Get	/cart	현재 로그인한 사용자의 이메일 정보를 이용하여 장바구니의 상품 목록을 조회합니다.
Patch	/cartProduct/{cartProductId}	장바구니 상품의 수량을 변경 요청합니다.
Delete	/cartProduct/{cartProductId}	상품 삭제 요청에 대한 컨트롤러를 구현합니다.
Post	/cart/orders	장바구니 상품 중에서 선택된 상품들에 대한 주문을 요청합니다.

Rest 지원 자바 API

경량화된 REST 방식의 웹 애플리케이션 구현을 지원하는 자바 API에는 다음과 같은 항목들이 있습니다.

항목	설명
JAX-RS	Java EE(Enterprise Edition) 표준의 일부로서, REST 아키텍처 스타일을 따르는 웹 서비스를 구현하고 제공하는 데 사용되는 API입니다.
RestController	스프링에서 사용하는 컨트롤러 입니다.

JAX-RS의 특징/기능

항목	설명
간편한 개발	RESTful 웹 서비스를 개발하는 데 필요한 다양한 기능을 제공하며, 이를 통해 개발자는 간단하고 효율적으로 RESTful 서비스를 개발할 수 있습니다.
어노테이션 기반 프로그래밍	어노테이션을 사용하여 자원을 매핑하므로 복잡한 XML 구성 파일 없이 간단하게 RESTful 서비스를 구현할 수 있습니다.
HTTP 지원	HTTP 프로토콜을 지원하며, GET, POST, PUT, DELETE 등의 HTTP 메서드를 사용하여 자원을 조작할 수 있습니다.
자동으로 데이터 변환	자동으로 Java 객체와 JSON 또는 XML과 같은 다양한 형식의 데이터 간 변환을 처리할 수 있습니다. 이를 통해 개발자는 비즈니스 로직에만 집중할 수 있고, 데이터 변환 작업을 간소화할 수 있습니다.
내장된 서버 구성 옵션	내장된 서버 구성 옵션을 제공하여 개발자가 간단한 설정을 통해 서비스를 배포하고 실행할 수 있습니다.
확장 가능성	확장 가능한 아키텍처를 제공하며, 필요에 따라 사용자 정의 기능을 추가하고 확장할 수 있습니다.